

Jamming Survival — On-Device Mesh Agent

Merged Design Document (authoritative)

Project	Jamming Survival — On-Device Mesh Agent (EAG V3 Capstone, Project 9)
Authors	Prateek Mohan Garg · Jatin Pahuja
Document	Merged HLD + detailed design
Version	1.0 — supersedes HLD v0.6 and DESIGN v1
Status	design frozen for build · 4-week delivery plan

How this document was produced. It takes Jatin's HLD (v0.6) as the authoritative spine — its EAG-V3-platform harness, its mapping to the five required deliverables, its frozen minimal config, its deterministic-predicate evaluation — and folds in the deeper technical design from the companion DESIGN doc: the discrimination *physics* behind the features, DAgger + cost-sensitive + calibrated-abstention training, the shared feature layer and int8-equivalence gate, the safety-by-construction mechanics, and the corrected RF-hardware story. The 14-week schedule is replaced by a **4-week calendar plan (~8 person-weeks, two people)** in §12.

1. Introduction

1.1 Purpose

This document is the high-level and detailed design for an on-device agent that keeps a drone's mesh radio link alive while that link is under attack, with no ground link available to ask for help. It defines the system boundaries, the components, the data that flows between them, the decision loop the agent runs, the training pipeline that produces the agent, and the evaluation harness that grades it. It is written to be implementable by a two-person team in four weeks and to be defensible to an examiner: every claim of "ground truth" traces back to a peer-reviewed open-source simulator (ns-3), not to code we wrote.

1.2 Scope

In scope

- A discrete, function-calling agent that runs on the drone (the **student**).
- The **harness** — our own agent loop, run as a *capstone profile* of the reused EAG V3 platform (S17Code + glc_v5). No third-party agent framework (LangChain, CrewAI, ...): the planner, capability registry, run journal and eval machinery are the platform's, gated to what this project needs (§5.3).
- A **Tier 1 simulation** (ns-3) that produces the attack corpus and the ground truth the agent is graded against — the stock OLSRV1 mesh, channel/fading models, the jammers, and the routing-adaptation interface (§6.4). ns-3 is vendored, pinned, and wrapped as the `run_sim` / `read_result` capabilities the harness calls (§6.6).
- A **classical deterministic diagnosis baseline** the agent is compared against.
- A **teacher→student pipeline**: a large model runs in simulation, its situation→action traces are captured, and a small model that fits on the drone is distilled from them. The gap between teacher, student and classical baseline is measured.

- A **task set** (data files the reviewer owns) with **deterministic verifiers** — each verdict is a predicate over the tool's output plus the ground-truth oracle, never over the agent's prose (§10.1–10.2).
- A **mutation corpus** — deliberately broken inputs — and the checker-robustness number it produces (§10.4).
- **Raw run records** written to disk before any scoring, so the scorer can change and be re-run without re-running the model or the simulator (§10.5).
- A **verifier** scoring detection latency, classification correctness, false-positive rate, recovery time, packets lost, actions consumed, and survival.

Out of scope

- Re-implementing planner / capability / eval / journal machinery the EAG V3 platform already provides — we configure and gate it, we do not rebuild it.
- Designing or modifying the fast radio loop — MAC/PHY, carrier sense, ARQ, and the internal routing algorithm (MPR selection, hello/TC logic, path computation). We run ns-3's stock OLSRv1 as a black box and configure it; we do not re-engineer it.
- Free-form natural-language generation on the device.
- Multi-drone joint optimisation. Neighbours are cooperative but each runs its own agent.
- Physical flight dynamics beyond a position model.
- Network scale as a research problem — node count is fixed small on purpose.

Stretch (designed for, not required for a complete capstone)

- **Tier 2:** a real 2.4 GHz rig of ESP32 mesh nodes plus one ESP32 interferer.
- **Tier 3:** an RTL-SDR dongle + GNU Radio to observe the **sub-GHz fallback band** and confirm cross-band recovery and the refusal condition (§11.3 — corrected RF story).

1.3 Alignment with the required deliverables

The project must produce five things plus a refusal task. Each is satisfied by a named part of this design and reuses a specific part of the EAG V3 platform rather than being built from scratch.

#	Required deliverable	Where	Reused from the platform
1	Your harness — own loop, no agent framework	§5.3	<code>proofs/harness.py</code> ; <code>planner.py</code> + <code>runtime.py</code> + <code>core/live_graph/</code>
2	The open-source tool, wrapped (<code>run_sim</code> , <code>read_result</code>), vendored + pinned	§6.6	<code>capabilities.py</code> (arg-contract validation); <code>coding/exec.py</code> (sandboxed runner)
3	A task set with verifiers — predicate reads tool output, never prose	§10.1–10.2	<code>evals/tasks.py</code> ; <code>score_contract</code> predicate pattern
4	A mutation corpus — broken inputs, fraction caught reported	§10.4	<code>evals/pairs.py</code> (TP/FP sweep)
5	Raw run records — to disk before scoring, scorer swappable	§10.5	<code>proofs/harness.py::Proof.finish()</code> ; <code>p4_trace_export</code>
—	Refusal task	§2, §7.6, §10.1	<code>proofs/p3_denial_of_wallet.py</code> (hard controller stops a runaway loop)

The one mismatch. The platform's agent is an LLM planner proposing a graph frontier each round; the capstone's deployed agent is a tiny classifier + policy head with **no LLM** (§9). So the platform maps onto the harness, the

teacher-side, and the evaluation infrastructure (offline, in the lab) — not onto the on-drone student, which is a separate artifact the harness evaluates.

2. Problem statement

The mesh is failing. The drone must work out **why** and act, with no ground link to ask for help. The hard part: several very different causes look almost identical from inside the radio.

The primary goal is **not** merely detecting jamming. It is **avoiding incorrect recovery actions** — above all, declaring "jamming" when the real cause is **fading**. That false positive has a real cost: the drone hops channels, loses the mesh, and *creates* the outage it was trying to avoid. The verifier weights this error the most heavily.

There is also a **refusal case**: broadband jamming across every available channel with no line of sight to any peer. There is no escape. The only correct action is to declare the link unrecoverable and execute the failsafe — return-to-home on the last known good position. An agent that hops channels forever has failed, and in the real world would fly until the battery ran out.

What the drone sees	What it might actually be
Delivery collapses, noise floor high on all channels	Barrage jamming
Delivery collapses on one channel only	Spot jamming
Delivery "fine" but everything is a retransmission	Reactive jamming (fires on your own TX)
Link up, no traffic from one peer	The peer died
Intermittent, correlated with distance/position	Fading & multipath — no attacker
Collisions rising with load	Congestion
Collisions without high local load	Hidden terminal

3. Goals and non-goals

Goals. Correctly classify the cause into {barrage / spot / reactive / sweep jamming, fading, dead peer, congestion, hidden terminal}; keep the false-positive rate low (require a positive distinguishing test before declaring jamming); recover quickly with the fewest costly actions; recognise the refusal case and execute the failsafe instead of thrashing; fit on the drone (small enough, fast enough, no GPU, no network); beat the classical baseline on the cost-weighted scorecard and report the teacher→student gap honestly.

Non-goals. Beating a human RF engineer; handling attackers outside the corpus (documented as a limitation); real-time signal-level DSP on the device; modifying OLSR's internals; demonstrating network scale.

4. Assumptions and constraints (frozen)

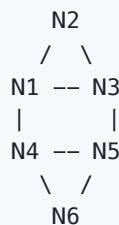
ID	Assumption
A1	Tier 1 (simulation) is mandatory and a complete capstone alone. Tier 2/3 are stretch.
A2	Ground truth comes from ns-3's own scenario configuration, not from code we wrote.
A3	The fast radio loop stays classical (stock ns-3 MAC/PHY/ARQ/OLSRv1). The agent runs at ~1 Hz decision cadence on top.
A4	The agent is a function caller over a fixed API (§6). Which function, which arguments, why. No free-form generation on the device; only the controller touches ns-3/radio objects.
A5	On-device student = a lightweight function-calling model: a classifier (posterior over causes) + a test planner + a policy head.
A6	On-device budget: no GPU, no network, footprint < 512 KB , inference < 20 ms , deterministic. Target a small ARM core; at the low end it must also fit an ESP32-S3 (~512 KB SRAM) for Tier 2.
A7	Teacher is a large instruction-tuned model, function-calling interface, offline, simulation only , never on the drone, never in the deployed loop.
A8	Radio context: 8 logical RF channels , one active channel per node, hopping supported, plus a low-rate LoRa fallback on a separate sub-GHz band .
A9	Mesh routing: OLSRv1 in ns-3, unmodified. batman-adv is at most an optional comparative experiment.
A10	Each incident has a hard action budget (default 6 costly actions) and a wall-clock recovery timeout, both enforced by the controller. Exhausting either forces escalation, not another hop.
A11	<code>move()</code> is available but is the most expensive test. The refusal decision must be reachable from sensing alone — never dependent on movement.
A12	Timeline: 4 calendar weeks, two people (~8 person-weeks) — see §12. (Compressed from the original 14-week estimate by reusing the platform and freezing scope.)
A13	Single drone/agent node (default N1); 5 cooperative peer nodes that will transmit a probe on request but are not jointly optimised.
A14	The minimal viable research system is frozen before implementation : 6 nodes + 8 channels + OLSRV1 + ns-3 + 8 degradation scenarios + agent + diagnostic actions + recovery + verifier. This stops the project sliding into a generic MANET/SDR effort.
A15	The harness is a capstone <i>profile</i> of the platform, not a fork that deletes code. S17Code and glc_v5 are pinned submodules; unused subsystems are gated off (feature flag + CI-enforced import isolation).
A16	ns-3 is vendored and pinned to a commit; its Python bridge vendored; builds on a clean machine with one command; exposed as <code>run_sim</code> / <code>read_result</code> .
A17	Records first, scoring second . Every run is a complete raw record on disk before any predicate runs; scoring is a pure offline function over records.
A18	Every verdict is a deterministic predicate over tool output + oracle. It never reads the agent's prose or stated confidence.

5. System context

5.1 Actors and external interfaces

Actor / interface	Direction	Notes
Harness (EAG V3 platform, capstone profile)	runs tasks through the agent loop, writes records	§5.3. Not on the drone.
Simulation environment (ns-3)	provides sensing, accepts actions	The "world"; reached via <code>run_sim</code> / <code>read_result</code> (§6.6).
Classical radio loop	provides raw counters, accepts coarse commands	Stock ns-3 in Tier 1; real firmware in Tier 2.
Ground-truth oracle	read-only, verifier only	The agent never sees it.
Teacher model	offline only	Function-calling interface, sim only; routed via <code>glc_v5</code> .
Classical baseline	consumes the same observations	Runs the same tasks for comparison.
Verifier (predicates + <code>score.py</code>)	consumes run records + ground truth	Deterministic; never reads agent prose (A18).
Operator / mission system	receives <code>declare_link_lost()</code> + failsafe	Out of scope beyond the interface.

5.2 Frozen network configuration (A14)



Parameter	Value
Total nodes	6 (1 drone/agent = N1 , 5 peers)
RF channels	8 logical channels , 1 active per node, hopping supported
Routing	OLSRv1, unmodified
Network	multi-hop wireless mesh, mobility supported
Channel model	normal + fading/multipath (Friis + Nakagami/Rayleigh + Jakes Doppler)
Fallback link	low-rate LoRa on a separate sub-GHz band
Ground truth	verifier only

Topology large enough to show direct-link failure, alternate routes, multi-hop recovery, node failure, congestion, channel-specific attacks and fading; small enough to run thousands of scenarios. Node count is fixed; growing it is a non-goal.

5.3 Harness architecture (reused EAG V3 platform)

The harness is a **capstone profile** of the EAG V3 platform. S17Code and glc_v5 are pinned git submodules; the project's own code lives in a `capstone/` package on top, exactly as repo-doctor sits on the same platform.

- **Reused unchanged:** the planner loop (`planner.py` — model proposes the next runnable frontier, Python validates authority/args/deps before anything executes), the capability registry and its argument-contract validation (`capabilities.py`), the sandboxed command runner (`coding/exec.py` — no shell, allowlist, workspace boundary, timeout, output cap), the durable run journal (`core/live_graph/`), the run harness (`proofs/harness.py`), and the eval loaders + threshold sweep (`evals/`).
- **New, in `capstone/`:** the ns-3 bridge and its `run_sim` / `read_result` capabilities; the Sense/Diagnose/Act capability contracts; the jamming predicates (built on `score_contract`); the mutation corpus + detection-rate sweep; the **classifier + policy-head student** and its **shared C feature layer** (§9.5); the **teacher pipeline**.
- **Gated off (Appendix D):** durable memory + faiss, A2A + grpc, the economics engine, channel adapters and voice. Present in source, disabled by `S17_PROFILE=capstone` , kept out of the capstone import graph by an **import-linter** contract that fails CI if anyone wires them in. Still tested by the platform's own full-profile CI.

Why gate rather than delete: the same platform carries forward to later projects, and nothing in the gated set runs on the drone, so deleting it relieves no on-device constraint.

6. Interfaces

The action space is small, deterministic, and split into Sense / Diagnose / Act plus a routing adaptation interface. Every call goes through the controller; the model never manipulates ns-3 or radio objects. The whole simulation is reached through two coarse capabilities the harness calls — `run_sim` and `read_result` (§6.6).

6.1 Sense API (read, free)

`get_rssi()` · `get_noise_floor()` (per channel) · `get_pdr()` (per link, rolling window) · `get_retry_rate()` · `get_spectrum()` (last snapshot: power vs channel) · `get_neighbor_status()` · `get_route_status()` . Derived context: offered load, own-TX-active flag, time since last own TX, position (x,y,z), neighbour count, action budget remaining.

6.2 Diagnose API (distinguishing tests, cost-ordered)

Function	Cost	What it separates
<code>spectrum_scan()</code>	1	jamming vs congestion (energy w/o valid packets vs valid packets); barrage vs spot vs sweep (one scan vs a sequence)
<code>listen_test()</code>	2	your receiver vs the channel — a neighbour transmits while you listen
<code>transmit_probe()</code>	2	your transmitter vs the channel — you transmit while a neighbour reports
<code>neighbor_probe()</code>	2	dead peer vs channel/routing — is a specific peer alive on any channel
<code>silent_listen()</code>	2	reactive vs non-reactive — stop own TX for T, re-measure noise & retries
<code>load_test()</code>	2	congestion vs jamming — drop offered load, see if PDR recovers
<code>channel_hop_probe()</code>	4	spot vs barrage — recover on a scan-clean channel, or confirm none exists (refusal input)
<code>mobility_test()</code>	8	fading vs attacker — <code>move()</code> ~30 m; link changes markedly ⇒ fading, persists ⇒ attacker

6.3 Act API (recovery)

Function	Effect	Cost class
<code>hop_channel(n)</code>	move the mesh to channel n (of 8)	costly — mesh disruption, -1 budget
<code>set_tx_power(p)</code>	change transmit power	mild
<code>reroute(via)</code>	prefer a route through a neighbour	mild
<code>change_tdma_slot(s)</code>	change TDMA slot / duty cycle	mild
<code>move(x,y,z)</code>	reposition the drone	expensive — time, energy, -2 budget
<code>fallback_to_lora()</code>	drop to the low-rate sub-GHz LoRa link	costly — throughput collapse, survivable
<code>declare_link_lost()</code>	declare unrecoverable, trigger failsafe	terminal — ends the episode

The controller records, for every call: the function, the arguments, the hypothesis it was acting on, and the **expected observable change** ("if this was spot jamming, PDR on the new channel should exceed 0.8 within 3 s"). That expectation is what §7.5's recovery verifier checks against.

6.4 Routing adaptation interface (agent ↔ OLSRv1)

The agent operates *above* OLSR. It does not modify the algorithm; it reads link/route state and requests recomputation. Reads: `get_neighbors()`, `get_route(dest)`, `get_link_quality(neighbor)`, `get_route_quality()`. Control: `request_route_refresh()`, `request_alternate_route(dest)`.

6.5 Agent → verifier: episode log

One JSON record per episode: `episode_id`, `scenario`, `agent`, `attack_onset_t`, `first_correct_classification_t`, a `classification_trace` (timestamped posteriors), the `tests_run` (with costs), the `actions` (with `budget_after`), `tests_before_correct_classification`, `recovery_t`, `packets_lost`, `actions_consumed`, `survived`, `declared_jamming`, `false_positive`. This is what the deterministic predicates (§10.2) read — never the model's prose.

6.6 Simulator capabilities (`run_sim` / `read_result`)

ns-3 is exposed to the harness as two coarse, contract-checked capabilities, declared in the platform's registry and executed through the sandboxed runner (allowlist entry: the pinned ns-3 binary):

Capability	Arguments	Returns
<code>run_sim</code>	<code>scenario</code> , <code>seed</code> , <code>config</code> (channels, topology, jammer, fading, onset, budget), <code>agent</code> (student teacher baseline none)	<code>run_id</code> , exit status, path to the raw record
<code>read_result</code>	<code>run_id</code>	the episode log (§6.5) + the run journal, read from disk

The fine-grained Sense/Diagnose/Act calls are the interface the agent uses *within* a running episode, carried over the ns3-gym / ns3-ai bridge at ~1 Hz sim time. They resolve to reads and writes on the simulation `run_sim` started — they are not separate processes. Because sim time is decoupled from wall time, a teacher that takes seconds to answer costs nothing in fidelity.

7. Agent design

7.1 The loop

```
detect anomaly
-> form competing hypotheses (belief over 8 causes + "unknown")
-> select the cheapest distinguishing test (max expected info-gain / cost)
-> execute test -> observe -> update belief
-> confident? act (recovery) : not confident? run another test
-> verify recovery
-> recovered? return to nominal : budget/timeout exhausted? failsafe
```

Three layers, only the smallest is learned:

- **L0 radio (classical, unchanged):** PHY · MAC · ARQ · OLSRv1. The agent never sits in this loop; it only reconfigures it.
- **L1 percept (fixed-point features):** ring buffers → EWMA, correlations, CUSUM, run-length histograms → the feature vector (Appendix A). **One C implementation, three call sites** (§9.5). Plus the anomaly gate.
- **L2 agent (student, ~1 Hz):** classifier → belief; test planner → which diagnostic; policy head → which recovery action. Wrapped in the hard safety envelope (§7.6).

7.2 Anomaly detector (the gate)

Lightweight, always-on. A two-sided CUSUM on short-window PDR against a slow baseline trips when windowed statistics leave their nominal band (PDR drop, retry-rate rise, noise-floor rise, or sustained zero-RX on a link):

$$S_t = \max(0, S_{t-1} + (\text{pdr_baseline} - \text{pdr}_t - k)), \quad \text{fire when } S_t > h$$

$k \approx 0.02$ (slack); h tuned for false-alarm rate $< 1/\text{hour}$ and detection latency < 300 ms under a step drop. Tuned for a low missed-detection rate; false triggers are cheap because the next stage is only a spectrum scan.

Detection latency is measured from onset, so the gate's latency counts against the headline metric — do not set h conservatively to flatter the false-alarm number.

7.3 Hypothesis catalogue

ID	Hypothesis	Attacker?	Signature before any test
H1	Barrage jamming	yes	PDR collapse on all links, noise high on all 8 channels
H2	Spot jamming	yes	PDR collapse, noise high on one channel only
H3	Reactive jamming	yes	PDR "fine" but retries huge; interference correlates with own TX
H4	Sweep jamming	yes	Per-channel collapse migrating across scans, roughly periodic
H5	Fading / multipath	no	PDR intermittent, correlated with position/time, noise normal
H6	Dead peer	no	Link up, zero RX from one peer, noise normal, beacons absent
H7	Congestion	no	Collisions/retries rise with offered load, noise normal, backoff helps
H8	Hidden terminal	no	Collisions without high local load; RTS/CTS or slot separation helps
—	unknown / abstain	—	evidence contradictory, stale, or out of range $\Rightarrow$ do not classify (§9.4, §10.4)

7.4 Why classification is possible — the discrimination physics

The passive statistics are a **many-to-one projection** of the hidden cause: barrage, a deep fade and congestion can all read as "PDR 0.1, retries 0.9". What resolves them is a set of *discriminative statistics* — each keyed to a confusion pair — plus, where those are ambiguous, an *intervention*. Every statistic below is computable on an ESP32 in fixed point.

Stat	Definition	Separates
S1 RSSI/PDR corr	<code>corr(RSSI_t, PDR_t)</code> over 5 s	fading vs jamming — under fading received power itself drops so $p \gg 0$; under jamming the frames we <i>do</i> get arrive at normal RSSI while PDR collapses, $p \approx 0$. The primary false-positive defence.
S2 SINR decomposition	$\text{SINR} \approx \text{RSSI} - \text{noise}$	a SINR collapse from the <i>numerator</i> falling = path loss/fading; from the <i>denominator</i> rising = external interference. Same SINR, opposite diagnosis.
S3 energy-w/o-preamble	fraction of busy time with no decodable preamble	congestion is other people's <i>packets</i> (≈ 0); jamming is <i>energy</i> (≈ 1).
S4 TX-conditioned noise Δ	$E[\text{noise} \mid \text{we TX'd} < \tau \text{ ago}] - E[\text{noise} \mid \text{silent}]$, $\tau \approx 2 \text{ ms}$	large $\Rightarrow$ reactive jammer — makes C3 detectable <i>passively</i> , before spending <code>silent_listen</code> .
S5 cross-channel variance	variance of PDR/noise across scanned channels	high $\Rightarrow$ spot/sweep; low & all-bad $\Rightarrow$ barrage; low & all-good $\Rightarrow$ not a channel problem.
S6 cross-link agreement	fraction of peers degraded	1-of-N $\Rightarrow$ that peer/geometry; all $\Rightarrow$ channel or own RX.
S7 fade-duration stats	histogram of below-threshold run lengths	Rayleigh fades are short and analytically bounded by Doppler; jammer on/off is square and long.
S8 self-motion coupling	<code>corr(PDR, displacement)</code>	fading tracks position; a fixed attacker does not — this is what <code>mobility_test</code> exploits.
S9 heartbeat gap	time since any frame from a peer	a jammed peer still tries; a dead peer emits nothing ever.
S10 load coupling	<code>corr(loss, own offered load)</code>	congestion scales with load; a barrage jammer does not care.

S1–S4, S7 and S10 are **passive**. A good feature layer resolves most episodes before the agent spends an action; the agent's job is to know when the passive evidence is ambiguous and which single experiment closes the gap. That is what keeps the action budget low.

7.5 Distinguishing-test matrix and policy head

Tests are cost-ordered; the planner picks the test with the highest expected information gain per unit cost given the current posterior (§9.3 makes this learned, not hand-computed).

Situation	Cheapest useful test	Reads as
jamming vs congestion vs fading	<code>spectrum_scan()</code>	energy w/o valid packets $\Rightarrow$ jamming; valid packets $\Rightarrow$ congestion; no excess energy $\Rightarrow$ fading/node loss
barrage vs spot vs sweep	<code>spectrum_scan()</code> xN	all hot $\Rightarrow$ barrage; one hot $\Rightarrow$ spot; hot channel migrates $\Rightarrow$ sweep
reactive vs steady	<code>silent_listen()</code>	noise/retries fall to nominal while silent $\Rightarrow$ reactive
your RX vs channel	<code>listen_test()</code>	neighbour heard cleanly $\Rightarrow$ RX fine
your TX vs channel	<code>transmit_probe()</code>	neighbour reports you clearly $\Rightarrow$ TX fine
dead peer vs routing vs jamming	<code>neighbor_probe()</code>	no peer on any channel + noise normal $\Rightarrow$ dead peer
congestion vs jamming	<code>load_test()</code>	PDR recovers when load drops $\Rightarrow$ congestion
spot vs barrage / refusal input	<code>channel_hop_probe()</code>	recovers on a clean channel $\Rightarrow$ spot; no clean channel $\Rightarrow$ refusal input
fading vs attacker	<code>mobility_test()</code>	link changes markedly with small move $\Rightarrow$ fading; persists $\Rightarrow$ attacker

Policy head (classification $\rightarrow$ action). Mostly a *validated lookup table*; learned corrections only where teacher traces disagree with the obvious rule.

Confident cause	Action	Rationale
Spot	<code>hop_channel(cleanest from scan)</code>	a clean channel exists; move to it
Sweep	<code>hop_channel()</code> ahead of the sweep; schedule further hops	stay in front of the jammer
Reactive	<code>change_tdma_slot()</code> / cut duty; <code>set_tx_power</code> down; <code>reroute()</code>	deny the trigger; do not hop — it follows your TX
Barrage, clean band elsewhere	<code>fallback_to_lora()</code>	trade throughput for a surviving link
Barrage, no escape, no LOS	<code>declare_link_lost()</code> + RTH	refusal case (§7.6)
Fading	<code>set_tx_power</code> up / small <code>move()</code> / <code>reroute(better link)</code>	never <code>hop_channel()</code> — hopping here is the expensive false positive
Dead peer	<code>reroute(via alternate)</code> ; else <code>declare_link_lost()</code> for that peer	routing, not RF
Congestion	<code>change_tdma_slot()</code> / backoff; <code>reroute</code> load	reduce contention
Hidden terminal	<code>change_tdma_slot()</code> / RTS-CTS separation; <code>reroute</code>	separate colliding transmitters

After acting, the recovery verifier watches PDR/retry for a recovery window: restored $\Rightarrow$ return to nominal; not restored and budget remains $\Rightarrow$ fold the failure back into the posterior ("hopping to ch 6 did not help" is strong evidence against spot) and continue; budget/timeout exhausted $\Rightarrow$ failsafe.

7.6 Failsafe and safety envelope — the refusal is *guaranteed*

The refusal must not depend on the model having learned it. Three mechanisms, increasing authority:

1. **Logit masking at inference.** Before the argmax, logits for unavailable actions are set to $-\infty$. If `hops_used == budget`, `hop_channel` cannot be selected *at all* — four lines of C on the ESP32. "Hop forever" is impossible by construction, not by training.
2. **Monotone escalation ladder.** Ordered by cost/reversibility; the agent moves down it but never back up once a rung is exhausted: `no_op` → `tx_power/tdma` → `reroute` → `hop_channel` → `move` → `lora` → `declare_link_lost`.
3. **A dead-man rule outside the model entirely** (`safety/failsafe.c`). The controller forces `declare_link_lost()` when any of these hold — **reachable from sensing alone (A11)**:
 - `spectrum_scan()` shows all 8 channels above the jamming-energy threshold, persistently;
 - `neighbor_probe()` fails on every channel tried, noise high on each (no RX-only/routing explanation);
 - no clean channel exists for a hop probe **and** the LoRa fallback band is also jammed or offers no reachable peer;
 - action budget exhausted (A10) with the link still down; or recovery timeout exhausted.

This runs whether or not the model produces output — including if inference hangs. The model can trigger RTH *early* (recognising hopelessness in 6 s instead of 20 — the skill we want); it can never *prevent* it. A unit test asserts the broadband-no-LOS scenario terminates in `declare_link_lost()` within a bounded number of steps and never emits an infinite hop sequence.

8. Classical deterministic baseline

A hand-written rule set, built in **Week 2 before the agent**, that the agent must beat on the cost-weighted scorecard. It uses the same Sense/Diagnose interface and the same episode-log format.

```
PDR↓ AND noise↑ AND all 8 channels affected          -> barrage
PDR↓ AND noise↑ AND exactly one channel affected      -> spot
PDR~ok AND retry↑↑ AND degradation only during local TX -> reactive
per-channel collapse migrates across scans           -> sweep
PDR intermittent AND noise normal AND corr. w/ distance -> fading
link up AND zero RX from one peer AND noise normal   -> dead peer
retry↑ AND rises with offered load AND noise normal  -> congestion
retry↑ AND local load low AND RTS/CTS helps           -> hidden terminal
```

The baseline gives three things: a sanity floor for the metrics, a control in the teacher-vs-student comparison, and a concrete answer to "did the learned agent actually add value over rules?" If the student does not strictly beat it — especially on the FP and refusal columns — nothing was learned.

9. Teacher → student pipeline

9.0 What runs where

Component	Where it runs	In the deployed loop?
Teacher (large model)	offline only — lab, in simulation	No — used once, to generate traces
Trace store, distillation	offline only	No — produces the student bundle
Student (classifier + test planner + policy head)	on the drone	Yes — the only learned model there
Agent controller, Sense/Diagnose/Act adapters	on the drone	Yes
Verifier, ground-truth oracle	offline only	No — grading, never visible to the agent

The pipeline's output is the **student bundle (< 512 KB)**, flashed onto the drone. After that the drone is fully self-contained: no teacher, no network, no ground link.

9.1 Teacher

Runs only in simulation, only offline. Given the **same observation vector the student will see (no ground-truth access)** plus the API definitions, and asked to choose one call and state why. Output constrained to the fixed function-calling schema so traces are directly usable as student targets.

Critical rule: the teacher never sees ground truth. If it does, its traces encode a capability the student can never have, and the distillation silently teaches the student to guess. The teacher's advantage must be *reasoning*, not information.

Two additions over single-shot prompting, both cheap and both improving trace quality:

- **Self-consistency:** sample `k = 5` at temperature 0.7; keep the majority action and mean posterior. Disagreement among the 5 marks genuinely ambiguous states — upweighted in training.
- **Rejection sampling:** score every teacher episode with the verifier and **discard the ones the teacher got wrong**. We distil a *filtered* teacher, which is meaningfully better than the raw teacher — and is why the student can sometimes beat the teacher's own average score.

9.2 Trace schema (per step)

`episode_id`, `step`, `scenario_label` (ground truth — **training only**), `observation` (features + tests_so_far), `prior_posterior`, `teacher_choice` (kind test|act|declare, fn, args), `teacher_rationale` (free text — **teacher only; the student never generates it**), `outcome_delta`, `reward_components` (latency, fp, recovery, actions).

9.3 Student training

- **Classifier:** gradient-boosted trees (LightGBM) **or** a small MLP over the feature vector (windowed sense stats + results of tests run + context). Output = posterior over {H1...H8, unknown}. This is the **primary, low-risk student** — interpretable, trivially fits, fast to build.
- **Test planner:** a shallow decision tree / ordered rule set fitted to the teacher's test choices, keyed on posterior entropy and which tests remain.

- **Policy head:** the lookup table of §7.5, with learned corrections only where teacher actions disagree with the table.
- **Stretch student (S-neural):** a small **int8 temporal CNN** over the observation window that captures sweep periodicity, fade-run structure and reactive TX-correlation *in the model* rather than in hand-engineered features. Kept as an ablation and a "how much does the temporal model buy?" result — **not on the critical path** (§12 de-scope levers).

Three training refinements folded in from the depth doc:

1. **Belief head trained on free ns-3 labels.** ns-3 already knows the true cause for every tick, so the *classification* head is trained by ordinary supervised learning on unlimited simulator labels. The teacher is needed only for what ns-3 cannot label — *which test, and when to stop investigating*. This removes ~70% of the teacher calls and is the single biggest reason a 4-week teacher budget is affordable.
2. **Cost-sensitive loss.** Train to minimise **expected operational cost**, not accuracy: $L_{hyp} = \sum_c C(y, c) \cdot p_c$, where C is the confusion-cost matrix (`train/cost_matrix.yaml`). The `fading` → `any-jamming` cells carry the largest cost (8–10) because the response is a channel hop that splits the mesh; every other confusion costs less. The matrix is a config file and is reported in the paper — a design choice reviewers can see and dispute.
3. **Dagger — not optional.** Pure behaviour cloning on teacher traces fails predictably: the student visits states the teacher never did and error compounds (great on held-out traces, collapses closed-loop). Fix, 2 rounds:

```
for r in 1..2:
  roll out student S_{r-1} in ns-3 (closed loop, fresh episodes)
  at each decision point the student reached, ask the teacher what it would do
  add (student's state, teacher's action) to the dataset; retrain -> S_r
```

Expect the largest single closed-loop improvement here. **Uncertainty-based Dagger** (query the teacher only where the student is unsure or the episode later failed) cuts teacher cost ~60% at little loss.

9.4 Calibrated abstention (the "unknown" class)

Rather than a hand-tuned confidence threshold, set the abstain operating point with **conformal prediction**: on a held-out calibration set, choose the posterior-confidence quantile that bounds the error rate at a target level. This gives the "unknown" class a principled guarantee and simultaneously (a) suppresses low-confidence declarations that would trip the FP metric and (b) is exactly the mechanism the mutation-robustness metric (§10.4) rewards — a checker that routes contradictory/garbage input to abstain instead of confidently classifying it. One temperature scalar and one threshold, both baked into the exported bundle.

9.5 One feature layer, three call sites (sim-to-real contract)

The L1 feature code is written **once**, in plain C99 with no dynamic allocation, in `percept/`, and compiled into:

- the ns-3 build (C++ shim),
- the ESP-IDF firmware (Tier 2),
- a Python extension (`cffi`) for offline recomputation from raw logs.

Normalisation constants are **fixed, hand-set, committed** (`percept/norm.h`) — the *same* constants in ns-3 and on the ESP32 — not learned from the training set. This is the single highest-leverage decision for sim-to-real: two feature implementations means the model trains on one distribution and deploys on another, and a week disappears into "why does the hardware behave differently." `contract/agent_contract.json`, `percept/norm.h` and `verify/metrics.md` are **frozen in Week 1** and are the handshake between the two work tracks.

9.6 Deployment and the int8-equivalence gate

Export path: PyTorch/LightGBM → ONNX / plain weight arrays → int8 → reference C (or MicroPython) inference; footprint < 512 KB. **Before anything ships**, run all validation windows through the offline model and through the on-device firmware and assert the argmax action agrees on $\geq 99.5\%$ and the posterior L1 distance is < 0.02 . Quantisation bugs are silent and look exactly like "the model doesn't generalise to hardware" — this gate catches them.

9.7 Three-way comparison (the result that answers the brief)

Run the identical held-out corpus (fixed seeds, never trained on) through (a) the classical baseline, (b) the teacher-in-the-loop agent, and (c) the distilled student. Report, per scenario family and overall: detection latency, classification accuracy + confusion matrices, false-positive rate, recovery time, packets lost, actions consumed, tests-before-correct, survival rate — and **every scenario where the student regresses**, listed explicitly. Two gaps are the thesis: **teacher→student small** (distillation kept most of the teacher) and **student»baseline** (the learning was necessary). Add an **S-on-ESP32** column once Tier 2 runs so the sim-to-real gap is visible in the same table.

10. Evaluation

10.1 Task set (frozen — 4 attacks, 4 non-attacks)

The scenario corpus *is* the task set. It lives in `data/tasks.jsonl`; each record is `{"id", "family", "config": {...}, "predicate": {...}}`. Nothing branches on the family label — it only groups the report. Each family is sampled across topologies, fading strengths, loads and attack-onset times to produce concrete instances (onset uniform in `[15 s, 30 s]` so there is a clean baseline first and an unambiguous `t_onset`).

Family	Attacker?	What it stresses
Barrage (all 8 channels)	yes	classification + escape logic + refusal
Spot (one channel)	yes	cheapest-test ordering, clean-channel hop
Reactive (fires only on your TX)	yes	silent-listen test, "don't hop" policy
Sweeping	yes	multi-scan reasoning, hop-ahead policy
Fading / multipath, no attacker	no	the false-positive trap
Dead peer	no	routing-vs-RF separation, neighbour probe
Congestion from a legitimate burst	no	load-change test, no false "jamming"
Hidden terminal	no	collision degradation, slot separation

Fidelity gate (Week 2): tune the fading family until the *marginal distribution of the passive features* is statistically indistinguishable from the barrage family (two-sample test). If fading is visibly easier than jamming, the false-positive number is a lie. This is a gate, not a nicety.

10.2 Verifiers (deterministic predicates)

Each task carries a predicate — a function that reads the episode log (§6.5) and the ground-truth oracle and returns pass/fail with a reason. **It never reads the agent's prose or stated confidence**. The predicate code is generic (`score_contract` pattern); the per-task input is declarative:

```
{ "classified_as": "reactive", "declared_jamming_allowed": true,
  "recovered_within_s": 5, "max_actions_consumed": 3,
  "must_survive": true, "must_end_in": null /* "declare_link_lost" for refusal
tasks */ }
```

A task passes only if every clause holds. A predicate that cannot be evaluated (missing/malformed field) is a **harness failure**, reported as such — it never silently becomes pass or fail.

10.3 Metrics (frozen before implementation)

Correct-classification rate (8-way + sub-type) · **false-positive jamming rate** (non-jamming classified as jamming; weighted highest) · detection latency (onset → first correct, held 3 decisions; **censored, not dropped**, when it never happens — report the censoring rate) · recovery time · packets lost · actions consumed (hops weighted) · tests-before-correct · survivability. Report mean ± 95% CI by **paired bootstrap over shared seeds**, per family and overall, tied to a git SHA. Report a second FP number — `FPR_acted` (agent actually issued a hop on fading) — alongside the belief-level FPR: the gap measures what the abstention threshold buys.

10.4 Mutation corpus

Tests whether the *checker* — the agent's input validation and its abstain path — notices a broken input instead of confidently classifying garbage. `data/mutations.jsonl`, one deliberate defect each, grouped by family: out-of-range (PDR 1.7, negative retry), contradictory (PDR 0.02 on a strong-RSSI, quiet-noise link), stale/frozen (identical sense values for 30 s), truncated (scan returns 3 of 8 channels), NaN/null, degenerate jammer config, duplicated event.

Metric — mutation-detection rate: fraction of mutated inputs the checker flags (raises "input invalid" / routes to abstain) rather than classifying, per family and overall. The complement matters too: **false-flag rate on the clean task set must stay near zero** — a checker that flags everything scores 100% and is useless. Both come from `evals/pairs.py::sweep` (TP/FP at each abstain threshold); the operating point is chosen the same way §9.4 sets the classifier's. Mutation families are fixed before the checker is tuned, and a held-out mutation set is never used in development.

10.5 Raw run records

Every run (student, teacher, baseline, mutation probe) writes a complete record to `data/runs/<run_id>.json` **before any predicate executes:** `run_id`, `task_id`, `agent`, `config` (exact scenario + seed), `platform` (s17code/glc_v5/ns3 commits), `episode_log` (§6.5), `journal` (the live-graph tape), `ground_truth`. Scoring is a separate offline pass (`score.py <runs-dir> <tasks-file>`). Changing a threshold, adding a metric, or fixing a predicate bug means re-running `score.py` over existing records — never re-running the model or ns-3. This is the platform's `p4_trace_export` property applied to the whole evaluation.

10.6 Pass thresholds (initial targets, calibrated Week 1–2)

Metric	Target
Classification accuracy (8-way)	≥ 85% on held-out
False-positive jamming rate	≤ 5%
Refusal-case handling	100% terminate in <code>declare_link_lost()</code> + RTH, no infinite hopping
Median recovery time (recoverable)	≤ 5 s
Median actions consumed (single-cause)	≤ 2
Mutation-detection rate	≥ 80% overall, false-flag ≤ 3% on clean set
Student vs teacher survival	within 10 pts
Student vs classical baseline	strictly better on cost-weighted score

10.7 Ablations (each answers a question a reviewer will ask)

Remove `rss_i_pdr_corr` (is S1 really what prevents the FP?) · remove `energy_no_preamble` (how much does jamming-vs-congestion depend on it?) · **passive-only, no epistemic actions** (how much is the *agentic* part worth over a classifier?) · no `silent_listen` (can reactive be caught passively via S4 alone?) · **BC-only vs +DAgger** (compounding error) · plain CE vs the cost matrix · fp32 vs int8 · window T = 4/8/16/32 · teacher traces halved (data-efficiency of distillation) · teacher-with-ground-truth (shows why we forbade it — should collapse).

11. Technology stack

Layer	Choice
Harness / platform	EAG V3 platform, pinned submodules — S17Code (<code>planner.py</code> , <code>capabilities.py</code> , <code>runtime.py</code> , <code>core/live_graph/</code> , <code>proofs/harness.py</code> , <code>evals/</code>) in a capstone profile
Teacher routing	glc_v5 gateway — ClaudeCLIProvider (offline, teacher pipeline only)
Profile & gating	<code>config/features.toml</code> (<code>S17_PROFILE=capstone</code>); <code>pyproject.toml</code> optional-deps so <code>faiss/grpcio/a2a-sdk</code> are opt-in; <code>import-linter</code> contract; per-profile CI smoke test
Simulator	ns-3 (vendored, pinned) — wifi, spectrum, propagation (Friis + Nakagami/Rayleigh + Jakes), olsr, mobility, energy; wrapped as <code>run_sim</code> / <code>read_result</code>
Custom ns-3 modules	<code>JammerApp</code> (barrage/spot/reactive/sweep — reactive via energy detection on an interferer node), <code>GroundTruthOracle</code> , <code>AgentBridge</code>
Agent ↔ sim bridge	ns3-gym / ns3-ai — per-step Python loop at ~1 Hz sim time
<code>capstone/</code> package	jamming capabilities, predicates (<code>score_contract</code>), mutation corpus + sweep, classifier + policy-head training — Python 3.11: <code>numpy</code> , <code>pandas</code> , <code>scikit-learn</code> / <code>LightGBM</code> , <code>matplotlib</code>
Percept (features)	one C99 library , no malloc → ns-3 shim + ESP-IDF + <code>cffi</code> (§9.5)
Student export	ONNX / plain weight arrays; reference C or MicroPython inference; < 512 KB , int8-equivalence gate
Run records / scoring	JSONL to <code>data/runs/</code> ; <code>score.py</code> as a separate offline pass
Optional Tier 2	ESP32-S3 mesh nodes (painlessMesh / ESP-NOW) + 1 ESP32 interferer; student on an ESP32-S3
Optional Tier 3	RTL-SDR + GNU Radio on the sub-GHz fallback band (§11.3)
Repo	Python + C/C++, <code>pytest</code> , <code>import-linter</code> , scenario/task/mutation configs as JSONL + YAML

11.1 Note on the on-device model form (why no LLM)

The smallest useful LLM at int4 (~25 MB+) exceeds an ESP32-S3's flash many times over, and streaming weights from PSRAM costs hundreds of ms per token. But the output is not text — it is one action of ~7, a few discrete arguments, and a posterior over 8 causes. So the student is a **constrained function-caller** (classifier + planner + policy head): the same "which function, which arguments, why" job an LLM tool-caller does, with the grammar enforced by the architecture instead of by decoding. The report states this with the arithmetic — it is the constraint that justifies the whole design.

11.2 Tier 2 — the ESP32 rig (stretch)

Five ESP32-S3 mesh nodes (node 0 = the "drone" running the student), one ESP32 interferer. The student trained in ns-3 runs unchanged on node 0 — Tier 2 is a change of *channel model* (real RF), not of agent. Real sensing the ESP32 genuinely exposes: per-frame RSSI and `rx_ctrl.noise_floor` (the key enabler), `channel/rate/sig_mode`, MAC-ACK success/fail (PDR), channel hop (~2 ms), TX power. The one honest gap — raw CCA-busy / energy-without-preamble (the S3 feature) is not directly exposed; approximate it as *noise elevated and decoded-frame rate near zero*, and **compute the feature the same coarse way in the simulator** so the model never trains on information the hardware cannot provide.

11.3 Tier 3 — RTL-SDR on the sub-GHz fallback band (corrected)

A stock RTL-SDR (R820T2) tunes ~24 MHz–1.7 GHz, so it **cannot see the 2.4 GHz attack band** — but it is excellent below ~1.7 GHz, which is exactly where the **LoRa sub-GHz fallback** lives (433 / 868 / 915 MHz, up to ~1 GHz). Use it there:

- **Make the cross-band fallback observable.** `fallback_to_lora()` hands off from 2.4 GHz Wi-Fi to a sub-GHz LoRa link. With the RTL-SDR watching the fallback band you can see whether it is clean before/after the handoff, and — critically — **verify the refusal condition** "the LoRa fallback band is *a/so* jammed or has no reachable peer" with a real measurement rather than an assumption.
- **A controlled refusal experiment:** put a cheap 433 MHz / second-LoRa transmitter spamming the fallback band, confirm the jammer PSD on the RTL-SDR, and check the agent correctly *refuses* rather than failing into a jammed band.
- **Two honest caveats.** (1) *Channel hop ≠ band switch*: hopping among the 8 Wi-Fi channels is one radio retuning within 2.4 GHz; going sub-GHz is a **hardware handoff** to the separate SX1278 LoRa module (the ESP32 Wi-Fi radio cannot tune there). (2) *RTL-SDR is receive-only* — it validates a band, it does not carry data. For watching the **2.4 GHz attack itself**, add a ~₹150 **nRF24L01+** (its RPD register sweeps 2.400–2.525 GHz) or a spare ESP32 in promiscuous mode. Buy both: nRF24L01+ for the attack picture, RTL-SDR for the fallback band.

Tier 2/3 BOM (stretch): 5× ESP32-S3-DevKitC-1 (~₹600 ea), 1× ESP32-WROOM (jammer, ~₹400), 2× nRF24L01+ (~₹150 ea), 2× SX1278 LoRa (~₹350 ea), 1× RTL-SDR Blog V4 (~₹2,500), powered USB hub + cables + antennas (~₹1,500). ≈ ₹9,000 total for a rig that does everything the brief asks.

12. Project plan — 4 weeks, ~8 person-weeks

This is the plan we present. Four calendar weeks, two people working in parallel (≈ 8 person-weeks total). The original 14-week estimate is compressed by three levers, each of which must actually hold or the plan slips:

1. **Reuse, don't rebuild.** The harness, planner, capability registry, sandboxed runner, run journal and eval machinery are the EAG V3 platform's (§5.3). We configure and gate them; we do not write them. This is where most of the 14→4 compression comes from.
2. **Freeze scope on day one (A14).** 6 nodes, 8 channels, OLSRv1, 8 scenarios, one primary student. No new protocols, no scale study, no second simulator.
3. **Two parallel tracks that meet at frozen interfaces.** `contract/agent_contract.json`, `percept/norm.h` and `verify/metrics.md` are frozen at the end of Week 1, after which the **World track** (Prateek) and the **Agent track** (Jatin) proceed independently and only re-integrate at the weekly gate.

Track ownership. Prateek → the world (ns-3, RF, attack corpus, ground-truth oracle, fidelity gate, optional hardware). Jatin → the agent (feature layer, baseline, classifier, test planner, policy, distillation, export). **Both** → the teacher pipeline, the verifier, and the report.

12.1 Week-by-week

Wk	Prateek — World track	Jatin — Agent track	Joint / gate
1 Foundation	Pin S17Code + glc_v5 submodules; capstone profile + <code>features.toml</code> + import-linter + green per-profile CI; vendor & pin ns-3; <code>run_sim / read_result</code> stubs through the sandboxed runner. ns-3 world: 6 nodes, 8 ch, OLSRV1, traffic, mobility; one barrage jammer breaks a link ; ground-truth oracle logging.	Feature layer (<code>percept/</code> , C99 + cffi); Sense/Diagnose adapters; teacher prompt + tool loop returns valid schema-conformant JSON on a hand-built state.	FREEZE <code>agent_contract.json</code> , <code>norm.h</code> , <code>metrics.md</code> . One live episode: sim → agent stub → action → record on disk.
2 World + baseline + verifier	All 8 scenarios (4 attack + 4 non-attack); reactive jammer (energy-triggered interferer); fading tuned to match barrage marginals — two-sample fidelity gate ; domain randomisation.	Classical baseline (§8); verifier — deterministic predicates + <code>score.py</code> ; records-first writer; mutation corpus + pairs.py sweep; task-set loader.	Gate P1+P2: ns-3 reproduces all 8 scenarios with logged truth; baseline scores all 8; verifier emits the full scorecard + mutation-detection number.
3 Teacher → student	Support DAgger rollouts (fresh closed-loop episodes on demand); domain-randomisation sweep; corpus for the teacher (~1.5k episodes).	Teacher rollouts (self-consistency + rejection sampling); belief head on free ns-3 labels ; BC → S_0 ; cost-sensitive loss ; DAgger ×2 → S_2 ; conformal abstention threshold; int8 export + equivalence gate .	Gate P4: student bundle < 512 KB, < 20 ms; runs the full corpus; beats baseline on the FP and refusal gates .
4 Eval + write-up (+ stretch)	Refusal-case hardening + unit test; ablation sweeps; stretch: Tier 2 ESP32 rig / Tier 3 RTL-SDR sub-GHz demo <i>if Weeks 1–3 held</i> .	Three-way comparison (baseline/teacher/student); ablations table; regression list; stretch: S-neural TCN as an ablation.	Final: headline table, teacher→student gap report, demo, <code>PLATFORM.md</code> , report.

12.2 Phase gates (must be true to proceed)

- **End W1:** contract + norm + metrics frozen; CI green on the capstone profile; one episode round-trips sim→agent→record.
- **End W2:** all 8 scenarios with ground truth; fidelity gate passed (fading ≈ barrage on passive marginals); baseline scored; verifier + mutation number emitted.
- **End W3:** student < 512 KB / < 20 ms; int8-equivalence ≥ 99.5%; student beats baseline on FP
 - refusal.
- **End W4:** three-way table complete with every regression listed; refusal unit test passes at 100%; mutation-detection reported per family within the false-flag cap.

12.3 De-scope levers (what gets cut first if a week slips)

Pull these **in order**, top first, to protect the Tier-1 core and the required deliverables:

1. **Tier 3 (RTL-SDR)** → drop to a written design only.
2. **Tier 2 (ESP32 rig)** → drop to a written design + the int8-equivalence result in sim only.
3. **S-neural TCN** → drop; the LightGBM/MLP student is the deliverable, TCN becomes "future work".
4. **Dagger round 2** → keep round 1 (round 1 captures most of the gain).
5. **Composite/held-out attack families** → keep the 8 singles; note generalisation as future work.

What never gets cut (these *are* the capstone): the 8-scenario ns-3 world with ground truth, the classical baseline, the deterministic-predicate verifier, the mutation corpus, records-first scoring, one distilled student under budget, and the refusal case working at 100%.

12.4 Effort budget ($\approx$ 8 person-weeks)

Track	W1	W2	W3	W4	Person-weeks
World (Prateek)	1.0	1.0	1.0	1.0	4.0
Agent (Jatin)	1.0	1.0	1.0	1.0	4.0
Total					8.0

The plan assumes the platform builds cleanly on a pinned commit (the largest schedule risk — see §13) and that both people can commit roughly full-time for the four weeks.

13. Risks and mitigations

Risk	Impact	Mitigation
Platform doesn't build on the pinned commit / latent bug on our path	Lost days in W1 — the biggest 4-week risk	Pin to a commit with green CI for the capabilities we use; smoke-test <code>run_sim / read_result</code> on day 1; time-box to 2 days then fall back to a thin custom harness that satisfies the same 5 deliverables
Home-grown simulator not credible to an examiner	Core results dismissed	ns-3 is authoritative (A2); any fast sim is a re-validated approximation
Scope creep into a generic MANET/SDR project	Never finishes in 4 weeks	Scope frozen before coding (A14); node/channel/routing all fixed; de-scope levers (§12.3)
Reactive jammer hard to model in ns-3	Slip in W2	Prototype it W1; fallback = dedicated interferer node with energy-triggered TX
Fading → jamming false positives (the graded error)	Fails the headline metric	Require a positive distinguishing test before declaring jamming; S1 (RSSI/PDR corr); cost-weighted loss; fidelity gate + fading-heavy validation
Pure BC collapses closed-loop	Student looks good on traces, fails live	DAGger is in the plan from W3, not an afterthought (§9.3)
Agent hops forever in the refusal case	Worst real-world failure	Failsafe is a controller rule + logit masking, not model reasoning (§7.6); hard budget/timeout; unit test
Learned agent adds nothing over rules	"Why not just the baseline?"	Baseline built first and reported alongside; student must strictly beat it
int8 quantisation silently changes behaviour	Hardware "doesn't generalise"	Equivalence gate $\geq 99.5\%$ before ship (§9.6)
Two feature implementations drift	Sim-to-real gap mysterious	Structurally prevented: one C99 file, three call sites (§9.5)
Teacher cost/availability	Pipeline stalls	Belief head uses free ns-3 labels; tiered sampling; uncertainty-based DAGger; all traces cached & versioned
Mutation metric gamed by a flag-everything checker	Meaningless number	False-flag rate capped on the clean set; families fixed before tuning; held-out mutation set
Reused platform reads as "borrowed a big thing"	Harness credit disputed	§1.3 maps every deliverable to a named module; capstone profile keeps the live import graph small; <code>PLATFORM.md</code> states active vs dormant
Hardware procurement/debugging (Tier 2/3)	Demo slips	Stretch only (§12.3 levers 1–2); Tier 1 alone is a complete capstone

14. Open questions (for the two of us to close in Week 1)

- Exact on-device compute target for the write-up (we assume small ARM; must also fit ESP32-S3).
- Decision cadence — is ~1 Hz right, or does a faster loop change the story?
- Action budget default (assumed 6) and recovery timeout.
- Which S17Code / glc_v5 commits to pin (default: latest green for the capabilities we use).
- Does the teacher route through glc_v5's ClaudeCLIProvider, or call the Claude CLI directly?
- Which node is the drone — fixed at N1 or varied across episodes?
- Is batman-adv wanted as a comparative experiment, or dropped (assumed dropped for 4 weeks)?
- Are Tier 2/3 in or out for the final submission (assumed stretch)?

15. Appendix A — feature vector

Windowed over the last N seconds, per link and aggregated; **normalised with the fixed constants in `percept/norm.h`** (§9.5). Grouped by the discriminative statistic each serves (§7.4):

- **Delivery:** PDR current / delta / variance; PDR CUSUM (gate statistic); worst-link PDR; PDR spread across links (S6); fraction of links degraded (S6); time since onset.
- **Signal:** RSSI current / delta / variance / min; RSSI slope; **RSSI–PDR correlation (S1)**; frames-seen count; **SINR estimate & delta-vs-baseline (S2)**.
- **Interference:** noise now / delta-vs-baseline / std; CCA-busy fraction; **energy-without-preamble (S3)**; foreign decodable-frame rate (S3); **TX-conditioned noise delta (S4)**.
- **MAC/ARQ:** retry-rate EWMA; retries-per-success; max consecutive TX fails; queue occupancy; offered-load; **loss–load correlation (S10)**.
- **Spectral memory:** scan age; fraction of channels bad; noise variance across channels (S5); current-channel rank; best-alternate-channel margin; **periodicity score** (sweep, S5).
- **Temporal:** **fade-run mean / p95 (S7)**; outage duty cycle; outage period estimate (sweep).
- **Self/platform:** own speed; displacement since onset; **PDR–motion correlation (S8)**; **max peer heartbeat gap (S9)**.
- **Bookkeeping:** hops / scans / actions used (fractions); time-in-episode; previous posterior (fed back); last-action one-hot; one-hot of tests run + their outcomes.

Appendix B — actions by cost

Cost class	Calls	Budget impact
Free	all Sense calls	none
Cheap	<code>spectrum_scan</code>	none
Mild	<code>set_tx_power</code> , <code>reroute</code> , <code>change_tdma_slot</code> , <code>listen_test</code> , <code>transmit_probe</code> , <code>neighbor_probe</code> , <code>silent_listen</code> , <code>load_test</code>	none
Costly	<code>hop_channel</code> , <code>channel_hop_probe</code> , <code>fallback_to_lora</code>	–1 each (hop probe –4 in planner cost)
Expensive	<code>move</code> , <code>mobility_test</code>	–2
Terminal	<code>declare_link_lost</code>	ends the episode

Appendix C — platform subsystem map (PLATFORM.md preview)

Subsystem	Module(s)	Capstone profile
Planner loop	planner.py , runtime.py	active
Capability registry + validation	capabilities.py	active
Sandboxed command runner	coding/exec.py	active (runs run_sim)
Run journal (the tape)	core/live_graph/	active
Eval loaders + threshold sweep	evals/tasks.py , pairs.py	active
Proof/run harness	proofs/harness.py	active (adapted)
LLM gateway	glc_v5 (ClaudeCLIProvider)	teacher-only
Adversarial nested validator	coding/validate.py	dormant
LLM-as-judge	evals/judge.py	dormant (our verdicts are deterministic, A18)
Durable memory + faiss	core/memory/	gated off (import-linter forbids in capstone/)
A2A + grpc	core/a2a/	gated off
Economics engine	economics/	gated off
Events / channels / voice	events/ , channel adapters	gated off

16. What "done" looks like

1. An ns-3 world that manufactures all 8 causes with ground truth, whose fading is *provably* as hard as its jamming (fidelity gate).
2. A deterministic-predicate verifier reporting detection latency, classification, the false-positive rate on fading, recovery time, packets lost, actions consumed and survival — per family, with CIs, tied to a code version — plus the mutation-detection number.
3. A **< 512 KB** function-calling student that runs in **< 20 ms** with no GPU/network, chosen over an LLM for reasons the report states with arithmetic.
4. The headline table: baseline $\ll$ student $\leq$ teacher, with the teacher $\rightarrow$ student gap small, the student-over-baseline gap large, and every student regression listed.
5. The refusal case working *by construction*: broadband-everywhere + no peer $\Rightarrow$ declare-lost $\Rightarrow$ RTH, within a bounded number of steps, at 100%.
6. **Stretch**: five ESP32s healing their mesh around a jamming sixth, with the RTL-SDR showing the sub-GHz fallback band and the agent's live belief resolving on screen.

The one-sentence thesis to defend: *the harness, the verifier's honesty and the discrimination physics do most of the work; the learned part is small on purpose, and we measure exactly how small it can be before it breaks.*